Prepared by Nidiana Rosado Hau
The University of Sheffield

# How to Build an Interactive Teaching Tool with AI

*A practical guide for educators — no coding knowledge required*

---

This guide explains how to use an AI assistant to build a free, interactive, pedagogically grounded web app for your students. You describe what you want in plain language. The AI builds it. You share a link. That is the entire process.

> **You do not need to know how to code.**
> *You need to know what good teaching looks like — and you already do.*

---

## What you are building

The result is a single HTML file — a webpage that works on any phone, tablet or laptop, with no app to install and no account to create. You host it for free on GitHub and share the link with your students. You can use google scripts u using offline (this will required all the link to pictures etc called locally)

The tool guides students through your topic step by step. It is not a video, not a slideshow, and not a PDF. It responds to what students do — it asks questions, gives feedback, unlocks the next step when understanding is demonstrated, and remembers where each student left off.

> **Real examples built this way:**
> - Hydrostatics Lab — four rotating activities covering density, pressure, free surface and Hare's tube
> - Wind Tunnel Lab — variable cross-section duct with live pressure charts from student data

*Both tools are free to use and adapt. See the open resource link at the end of this document.*

---

Prepared by Nidiana Rosado Hau
The University of Sheffield

# The process — four steps

Building a tool takes between 20 minutes and an afternoon, depending on how much content you include. Here is the complete process.

**1** **Write your prompt**
*Describe your topic, your students, the theory you want covered, and the pedagogical approach you want applied.*

**2** **Send it to an AI assistant**
*Paste your prompt into Claude (claude.ai) or ChatGPT. The AI generates the complete HTML file in one response.*

**3** **Save and upload**
*Copy the HTML, save it as a .html file, upload it to GitHub Pages. Your tool is live and shareable within minutes. Use it locally for trials first.*

**4** **Fix anything that is wrong**
*If something is missing or incorrect, describe the problem to the AI in plain language. It fixes it. No rebuilding required.*

---

# How to write your prompt

Your prompt has three parts. You do not need to write them as separate messages — one combined message works well. The structure below shows what to include and why.

## Part 1 — Your topic and your students

Start with a single sentence. Describe what subject you are teaching and what your students already know. This tells the AI the right level of explanation to use.

> **Example:**
> `"Build a single-file interactive HTML teaching guide for pump hydraulics in civil engineering. The audience is mid-course undergraduate students who already know basic fluid mechanics — Bernoulli and pipe friction losses."`

*You can adapt this sentence for any subject. Replace the topic, replace the level. Everything else in the prompt stays the same.*

Prepared by Nidiana Rosado Hau
The University of Sheffield

## Part 2 — The pedagogical approach

This is the most important part. Copy and paste this block into every prompt you write. It tells the AI how to teach, not just what to teach. You do not need to change it.

```
Apply these pedagogical principles to every design decision:

// Reveal information step by step, never all at once.
// Students see only what they need for the current step.
Cognitive Load Theory: reveal information step by step, never all at once.
Students should only see what they need for the current step.

// Support fades as students gain confidence.
Scaffolding: early steps give explicit guidance. Later steps ask students
to apply and reason independently.

// Words and formulas together, never apart.
Multimedia Learning: when you show a formula, explain it in plain language
in the same place. Never separate the equation from its meaning.

// Information at the moment it is needed.
Just-in-time information: formulas, theory and safety notes appear at the
exact moment they are relevant, not in a preamble at the start.

// Feedback that teaches, not just judges.
Immediate feedback with reasoning: a wrong answer must explain why it is
wrong, not just say incorrect. Input boxes stay open for retry.

// Students always know where they are.
Visible structure: students must always be able to see where they are and
what comes next. A step indicator at the top must always be visible and
allow free navigation so no student feels trapped.

// Usable by everyone.
Accessibility: high contrast colours, never colour as the only signal,
minimum 14px font, mobile responsive.
```

## Part 3 — Your content

Now describe what you actually want students to learn. Include the topics, the theory, any specific formulas, and the kinds of questions you want. The more specific you are here, the better the result.

**Example — pump hydraulics:**

```
"Cover three topics as three separate activities:
Activity 1: Pump types — centrifugal, axial, mixed flow.
Activity 2: The H-Q characteristic curve and operating point.
Activity 3: Pumps in series and parallel.
```

```
For each activity include: a real-world challenge that frames why the topic
matters, the core theory with formulas, at least one multiple choice question
with targeted feedback for every wrong answer, and one question connecting
the topic to real engineering practice."
```

*You do not need to write out all the theory yourself. The AI knows your subject. You only need to tell it which topics to cover and what kind of questions to include.*

---

## The complete prompt — ready to use

Here is the full prompt combining all three parts. Copy it, change the first sentence and the content block at the bottom to match your subject, and paste it into any AI assistant.

```
Build a single-file interactive HTML teaching guide for [YOUR TOPIC].
The audience is [YOUR STUDENTS — describe what they already know].

Apply these pedagogical principles to every design decision:

Cognitive Load Theory: reveal information step by step, never all at once.
Students should only see what they need for the current step.

Scaffolding: early steps give explicit guidance. Later steps ask students
to apply and reason independently.

Multimedia Learning: when you show a formula, explain it in plain language
in the same place. Never separate the equation from its meaning.

Just-in-time information: formulas, theory and safety notes appear at the
exact moment they are relevant, not in a preamble at the start.

Immediate feedback with reasoning: a wrong answer must explain why it is
wrong, not just say incorrect. Input boxes stay open for retry.

Visible structure: a step indicator at the top must always be visible and
allow free navigation so no student feels trapped.

Accessibility: high contrast colours, never colour as the only signal,
minimum 14px font, mobile responsive.

Save all student inputs and progress to localStorage automatically.
Restore the full session on page refresh. Include a reset button.
```

Prepared by Nidiana Rosado Hau
The University of Sheffield

```
Single self-contained HTML file. Use MathJax CDN for equations.

Content:
[Describe your topics, theory, formulas and questions here]
```

## When something is wrong — how to fix it

The first result will rarely be perfect. Something may be missing, visually off, or not quite what you intended. This is normal — and easy to fix. You never need to rebuild from scratch.

### Describe the problem in plain language

Send a follow-up message to the same AI conversation describing exactly what is wrong. One problem per message works best. The AI updates the file and returns the corrected version.

#### Common problems and how to describe them

**Problem: equations showing as raw text like $N_s$ instead of rendered maths**

**Say this:**

```
"The equations are not rendering — they show as raw LaTeX text. After any
dynamic content is shown, call MathJax.typesetPromise() to force all
equations to render correctly."
```

**Problem: design is plain and visually dull**

**Say this:**

```
"Update the visual design. Use dark navy #0D1B2A for headers, teal #0EA5E9
as accent, amber #F59E0B for highlights. White card backgrounds with box-
shadow. Section blocks with a 4px coloured left border. Keep all content
and functionality unchanged."
```

**Problem: no step indicator at the top — students cannot see where they are**

**Say this:**

```
"Add a persistent step navigation bar at the top of every activity showing
all steps as numbered circles. Current step highlighted in teal, completed
steps show a tick, upcoming steps in grey. The bar must remain visible as
the student scrolls and allow free navigation between steps."
```

**Problem: a specific feature is missing**

**Say this:**

```
"[Name the feature] is missing from [where it should be]. Please add it.
Keep everything else unchanged."
```

**Problem: you want to show an image of your own equipment**

**Say this:**

```
"In Activity 1 Step 1, replace the placeholder image with this Google Drive
image URL: [paste your URL]. Display it at full width on desktop, full
width on mobile."
```

### You can also show the AI an example

If you are struggling to describe what you want, show it instead. Paste in a section of HTML from an existing tool that has the feature you need and say:

```
"Here is an example of the feature I want. Apply the same approach",
"to my tool in [location]. Keep everything else unchanged."
```

*The AI reads the example, understands the pattern, and replicates it. This is especially useful for complex interactive features like live charts or collapsible panels.*

---

## Saving the file and sharing with students

### Step 1 — Save the HTML file

When the AI generates your tool, the response will contain a large block of code starting with <!DOCTYPE html> and ending with </html>. To save it:

1. Copy everything from <!DOCTYPE html> to </html>
2. Open any plain text editor — Notepad on Windows, TextEdit on Mac
3. Paste the code and save the file with a .html extension, for example pump-hydraulics.html
4. Open the file in a browser (locally) to check it works before uploading

### Step 2 — Upload to GitHub Pages

GitHub Pages is free and permanent. Your tool will have a public URL you can share immediately.

**First time only — create your repository:**

1. Create a free account at github.com
2. Click the + icon and select New repository
3. Name it (e.g. teaching-tools), set it to Public, tick Add a README, click Create
4. Go to Settings → Pages → Source → select main branch → Save

**Every time you add a new tool:**

1. Open your repository on github.com
2. Click Add file → Upload files
3. Drag your .html file in, click Commit changes
4. `Your tool is live at: https://[username].github.io/[repo-name]/[filename].html`

*Name files with lowercase letters and hyphens — no spaces. For example: pump-hydraulics.html, concrete-mix-design.html. This keeps your URLs clean and easy to share.*

Prepared by Nidiana Rosado Hau
The University of Sheffield

# Master Keyword List

Use these keywords and phrases when writing prompts. Each entry describes a feature precisely enough for an AI to implement it correctly. Mix and match from different sections to build exactly the tool you need.

| Keyword / Feature | What it does |
|---|---|
| **Structure & Navigation** | |
| **Step-by-step guided sequence** | Content revealed one step at a time, reducing cognitive overload |
| **Visible progress bar with percentage** | Always-visible bar shows how far through the activity the student is |
| **Numbered step indicator at top** | Step pips (1→2→3→4) always visible and clickable — students are never lost |
| **Previous button to revisit steps** | Students can return to earlier steps without losing their work |
| **Return to main menu button** | Exit any activity and return to the activity selector at any time |
| **Activity carousel / rotation selector** | Landing page showing all available activities as selectable cards |
| **Completion card with congratulations** | Green card appears when an activity is fully finished |
| **Final summary with key equations** | Closing slide displaying all formulas covered, printable |
| **Interaction & Engagement** | |
| **Checkbox items that highlight green when ticked** | Visual confirmation that each item has been acknowledged |
| **All checkboxes required before proceeding** | Next button stays locked until every checkbox is ticked |
| **Radio button multiple choice questions** | Single-answer questions with clearly labelled options |
| **Input boxes for numerical data entry** | Students type in their own measurements directly into the tool |
| **Dropdown menus for identification** | Students select a fluid, material or category from a list |
| **Auto-calculation from student input** | Density, ratio or derived value calculated automatically on entry |
| **Real-time graph from student data** | Chart updates as the student fills in the data table |
| **Verification feedback on dropdown selection** | Correct / incorrect confirmation with explanation after selection |
| **Uncertainty input with check and hint reveal** | Student enters uncertainty value; hint appears if incorrect |
| **Feedback & Assessment** | |

Prepared by Nidiana Rosado Hau
The University of Sheffield

| Keyword / Feature | What it does |
|---|---|
| **Immediate right/wrong feedback on quiz** | Result appears the moment the student submits their answer |
| **Targeted explanation of why answer is wrong** | Not just 'incorrect' — the reasoning behind the correct answer is shown |
| **Correct answer revealed with full reasoning** | Student always leaves knowing the right answer and why |
| **Input box highlights red for wrong, green for correct** | Colour-coded visual signal integrated into the input field itself |
| **Retry allowed — input stays open after wrong answer** | Student can attempt again without reloading or losing context |
| **Neutral feedback for open-ended questions** | Self-assessed questions give a prompt rather than right/wrong |
| **Progress only advances after correct response** | Ensures understanding before moving forward — not just completion |
| **Just-in-Time Information** | |
| **Learning objective box at step start** | Challenge or goal is stated clearly before the student begins |
| **'Why this matters' contextual box** | Relevance to real engineering or practice is explained in context |
| **Formula block shown at point of use** | Equation appears at the step where it is needed, not upfront |
| **Safety warning with required acknowledgement** | Warning box cannot be bypassed — student must confirm they have read it |
| **Hint box appearing only after wrong attempt** | Help is available but does not short-circuit the thinking process |
| **Collapsible guidance panel (show/hide toggle)** | Instructor-level explanation available on request, hidden by default |
| **Background theory before procedure begins** | Conceptual grounding is given before the student picks up equipment |
| **Contextual reminder linked to previous step** | Cross-reference appears when a concept from an earlier step is needed |
| **Data & Visualisation** | |
| **Data recording table with input fields** | Structured table for systematic measurement entry |
| **Auto-calculated column from measurements** | Derived values (density, ratio, average) computed without manual arithmetic |
| **Live bar chart from student pressure data** | Chart renders from the student's own entered values |
| **Line chart comparing experimental vs theoretical** | Overlay of measured data against expected curve |

Prepared by Nidiana Rosado Hau
The University of Sheffield

| Keyword / Feature | What it does |
|---|---|
| **Colour-coded zones matching apparatus sections** | Contraction, throat and diffuser shown in distinct colours on chart |
| **Chart appears only after concept question answered** | Visualisation is a reward for demonstrated understanding |
| **Average calculation displayed dynamically** | Running average updates as more data rows are completed |
| **Cognitive & Pedagogical Design** | |
| **Prediction question before experiment** | Student commits to an expected outcome before collecting data |
| **Prediction feedback after experiment** | Student's prediction is revisited and compared to actual result |
| **Metacognitive question for self-assessment** | Prompts student to reflect on their own understanding and technique |
| **Discussion prompt with collapsible guided answer** | Open question for group discussion; guidance revealed on request |
| **Theory embedded at point of relevance** | Pascal's Law appears at the step where it explains the observation |
| **Equipment image spatially paired with procedure** | Image and instruction are integrated — not separated across pages |
| **Safety and procedure integrated, not separated** | Hazard information appears within the workflow, not in a preamble |
| **Orientation & Anxiety Reduction** | |
| **Step map always visible at top of screen** | Student always knows where they are and what comes next |
| **Step pips clickable to navigate freely** | Student is guided but never trapped — free navigation at all times |
| **Current step highlighted, completed steps marked** | Clear visual distinction between done, current and upcoming steps |
| **Total step count displayed at all times** | Student knows how many steps remain — no hidden workload |
| **Activity cards showing all options before starting** | Full overview of the session before committing to an activity |
| **Activity card marked green when complete** | Completed activities are visually distinguished from remaining ones |
| **Smooth scroll to top on step transition** | Page repositions automatically so student always starts at the top |
| **Data Persistence & Session Resilience** | |
| **Student inputs saved automatically to localStorage** | Every keystroke is saved — a page refresh never loses entered data |
| **Checkbox and quiz states restored on reload** | Ticked boxes and answered questions survive accidental refresh |

| Keyword / Feature | What it does |
|---|---|
| **Completed activity cards remain green after refresh** | Session progress is preserved across page reloads |
| **Progress bar and step position restored on reload** | Student returns to exactly where they left off |
| **Data cleared only intentionally** | Nothing is lost accidentally — only a deliberate reset clears data |
| **Optional reset button for shared devices** | A clearly labelled button allows a new student to start fresh |
| **Per-device, per-browser storage** | Each browser session is independent — suitable for shared lab computers |
| **♿ Accessibility & Inclusivity** | |
| **High contrast colour palette throughout** | Text and background combinations meet WCAG AA contrast ratios |
| **Colour never used as the only signal** | Error and success states always include text, not colour alone |
| **Minimum font size 14px for body text** | Readable without zooming on standard screens |
| **Mobile responsive layout** | Layout adapts cleanly to phone and tablet screen sizes |
| **Touch-friendly inputs minimum 44px height** | Buttons and fields are large enough for touch interaction |
| **Screen-reader friendly labels on every input** | Every input field has an accessible label for assistive technology |
| **⚙ Technical Features** | |
| **MathJax rendered equations from LaTeX** | Professional mathematical notation rendered in the browser automatically |
| **Google Drive image embedding** | Images hosted in Google Drive can be embedded directly by URL |
| **GitHub Pages hosting ready** | Single HTML file can be uploaded and live within minutes |
| **Single self-contained HTML file** | No server, no database, no dependencies — one file does everything |
| **Works offline after first load** | Once loaded, the tool works without an internet connection |
| **Print stylesheet for summary export** | Student can print a clean summary of their completed work |
| **No account or login required** | Students access the tool instantly — no registration barrier |

Prepared by Nidiana Rosado Hau
The University of Sheffield

# Hosting on GitHub Pages — Step by Step

GitHub Pages is free, permanent and requires no technical knowledge beyond the steps below. Your tool will have a public URL you can share with students immediately.

## First time setup

1. Create a free account at github.com
2. Click the + icon and select New repository
3. Give it a name (e.g. lab-tools), set it to Public, tick Add a README, click Create
4. Go to Settings → Pages → Source → select main branch → click Save
5. Your site URL will be: https://[your-username].github.io/[repository-name]/

## Uploading a new tool

1. Open your repository on github.com
2. Click Add file → Upload files
3. Drag your .html file into the upload area
4. Click Commit changes — the file is now live within 1–2 minutes
5. Share the URL: https://[username].github.io/[repo-name]/[filename].html

**Tip: Name your files clearly.**
Use lowercase with hyphens and no spaces — e.g. hydrostatics-lab.html, wind-tunnel.html. This keeps your URLs clean and easy to share.

Prepared by Nidiana Rosado Hau
The University of Sheffield

## Open resource — use and adapt freely

All tools developed using this approach are shared as open resources. You are free to copy them, adapt them for your own subject, and share them with colleagues.

> **GitHub repository with working examples:**
>
> `github.com/Nidiana/LabsDiscussion`
>
> The repository contains several examples. You can open either file in a browser to see a finished example, copy the HTML as a starting point for your own subject, or paste it into an AI and ask it to adapt the content for a different topic.

Last update 24/03/2026.